Generative AI

Types of AI

1. Weak AI (Narrow AI)
2. Strong AI (General AI)
3. Super AI (Artificial Superintelligence)

Weak AI (Narrow AI)

- Designed to perform a specific task only.
- Around 90% of AI systems in use today are weak AI.

Strong AI (General AI)

- AI that can perform any intellectual task like a human.

Super AI (Artificial Superintelligence)

- AI that can perform beyond human capabilities.

Generative AI

- A subset of Deep Learning (DL), which is itself a subset of Machine Learning (ML).
- Two types of Deep Learning:
    - Discriminative Deep Learning
    - Generative Deep Learning

Discriminative Deep Learning

- Input → Model → Output (prediction/classification).

Generative Deep Learning

- Generates new information from a previously trained model.

Main Uses of Generative AI

1. Text Generation
2. Image Generation
3. Code Generation

Text Generation

Includes:

- Question answering
- Problem solving
- Summarization
- Grammar checking
- Letter writing
- The models used for these tasks are LLMs (Large Language Models)

Examples:

- **GPT (Generative Pre-trained Transformer)**
- Developed by OpenAI

- **Gemini**
- Developed by Google
- **LaMDA (Language Model for Dialogue Applications)**
- Developed by Google
- **LLaMA (Large Language Model Meta AI)**
- Developed by Meta AI

Fine-Tuning

- The process of adjusting a pre-trained model to make it perform better for a specific task or requirement we want.
- It involves training the model further on a smaller, task-specific dataset.

Gemini

- Developed by Google

Steps:

- Create an API key

  Go to the notebook in Google Colab → select **Secrets** on the left side → enter the name → paste the API key into the value field → then hide it and allow the notebook access.

- The main free model used is **"gemini-2.5-flash."**
- response = model.generate_content(...) — generates a response to a question
  response — returns the output in JSON format
  response.text — returns the output as plain text

- **Text wrap** — used to display output in a clean and readable format, similar to how results appear when you ask Gemini.

```
from IPython.display import Markdown
import textwrap

def demo(text):
  return Markdown(textwrap.indent(text, '>'))
```

- For image based,
  a. Assign the qn to a variable
  b. Then use generate_content
  c. Eg

```
content = "Generate a description"
response2 = model.generate_content([content, img])
```

For creating a chatbox using Gemini:

1. Import genai
2. Set the Api Key
3. Configure the client
4. Select model
5. Use an infinite while loop to create the chatbox interaction

- To implement a chatbox in an ML project (or any project you build), store the user's question in a variable and use **if-else statements** to handle different types of inputs or responses.
- In Google Colab, we can securely hide the API key using **Secrets**, but in VS Code we typically store it in environment variables or a .env file instead of directly hiding it in the interface.
- Eg code for creating a Gemini ChatBox in VSCode:

```
import google.generativeai as genai
```

```
api_key = "your_api_key_here"
genai.configure(api_key=api_key)

model = genai.GenerativeModel("gemini-2.5-flash")

print("Google AI Gemini Chatbox")
print("Type 'exit' to quit\n")

while True:
    user_input = input("You: ")
    if user_input.lower() == "exit":
        print("Goodbye")
        break

    response = model.generate_content(user_input)
    print(response.text)
```

Customizable Chatbot

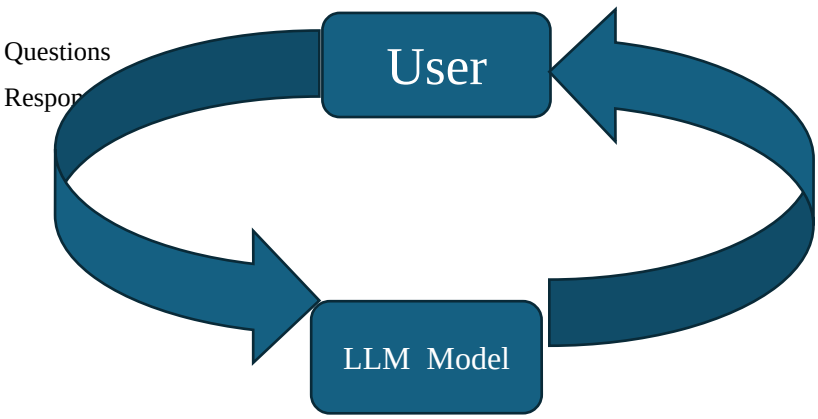
- Store the required information in a variable, then create a chatbot and instruct it to respond based on that information.
- **Drawback:** You need to define all the information manually.
- **Solution:** This limitation can be overcome using **RAG (Retrieval-Augmented Generation).**

RAG (Retrieval-Augmented Generation)

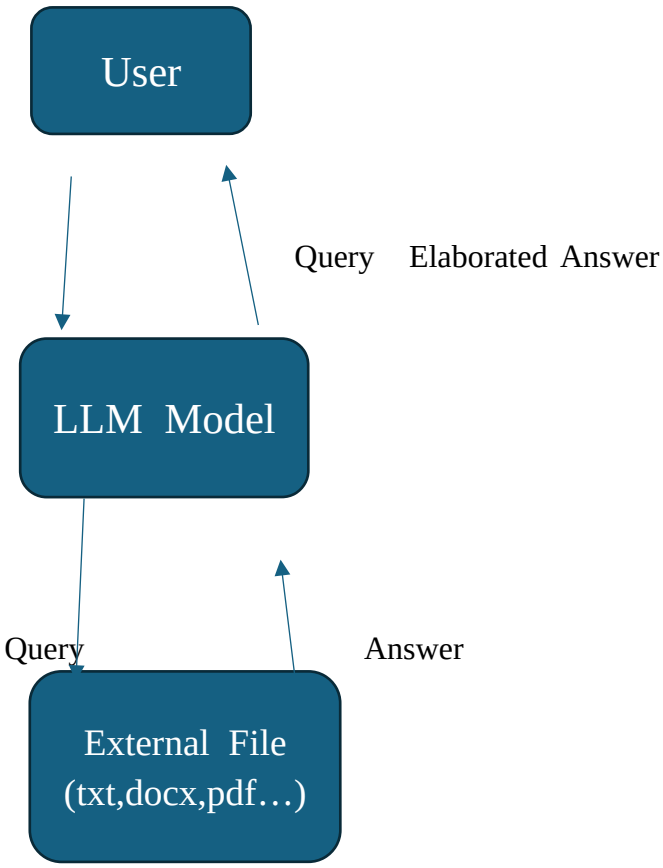
- A higher-level alternative to RAG is **fine-tuning**.
- A commonly used framework for implementing RAG is **LangChain**.

- It is mainly used to enhance the performance of an LLM by providing relevant external knowledge.

Normal LLM Model Works



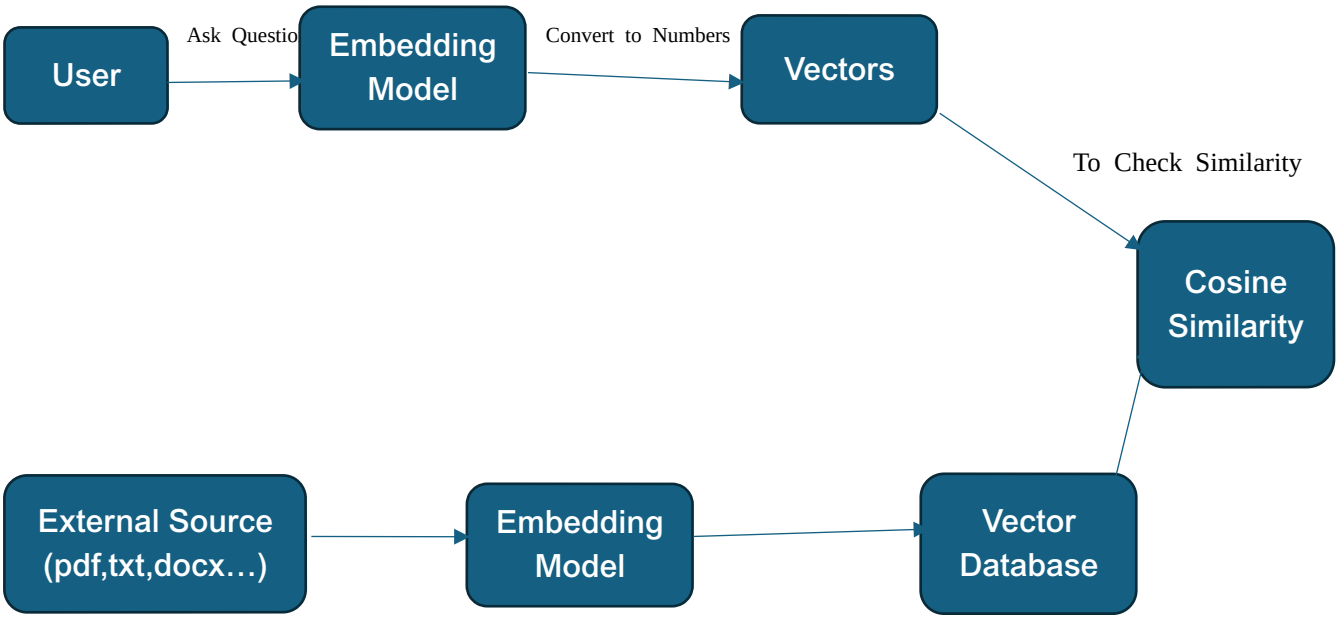
RAG Model Works



2 Components of RAG

1. **Retrieval:** Fetches knowledge from external sources.
2. **Generator:** Produces a response using the LLM based on the retrieved information.

Flowchart



- **Vector Database –** Stores vectors and their corresponding text chunks.
- **For example:**
    - Vector1 represents the text *"India won a trophy"* with embedding [0.1, 0.3, 0.4].

- Vector2 represents the text *"Brazil lost a game"* with embedding [0.58, 0.32, 0.2, 0.7].

  If the query is *"Who won a trophy?"*, the model computes the similarity between the query embedding and all stored vectors in the database. It then retrieves the chunk with the highest similarity score, which in this case would be *"India won a trophy."*

Cosine Similarity

- Cosine similarity measures how similar two vectors are by calculating the cosine of the angle between them.
- **Formula:**

  Cosine Similarity = $(A \cdot B)/(\sqrt{}¿A \sqrt{}¿ \times \sqrt{}¿B \sqrt{}¿)$
- **Expanded form:**

  Cosine Similarity = $(\Sigma\ A_iB_i)\ /\ (\sqrt{}(\Sigma\ A_i^2) \times \sqrt{}(\Sigma\ B_i^2))$, for i = 1 to n.
- The value ranges from -1 to 1:
  - 1 → exactly similar
  - 0 → no similarity (orthogonal)
  - -1 → completely opposite

Challenges or Drawbacks of LLM Models

- **Relevance of Information**

  LLMs may sometimes retrieve or generate information that is not fully relevant to the user's query.
- **Handling Incorrect Information**

  LLMs may not realize when information is wrong and can give incorrect answers that sound correct.
- **Hallucinations**

  LLMs may generate confident but factually incorrect or fabricated responses.

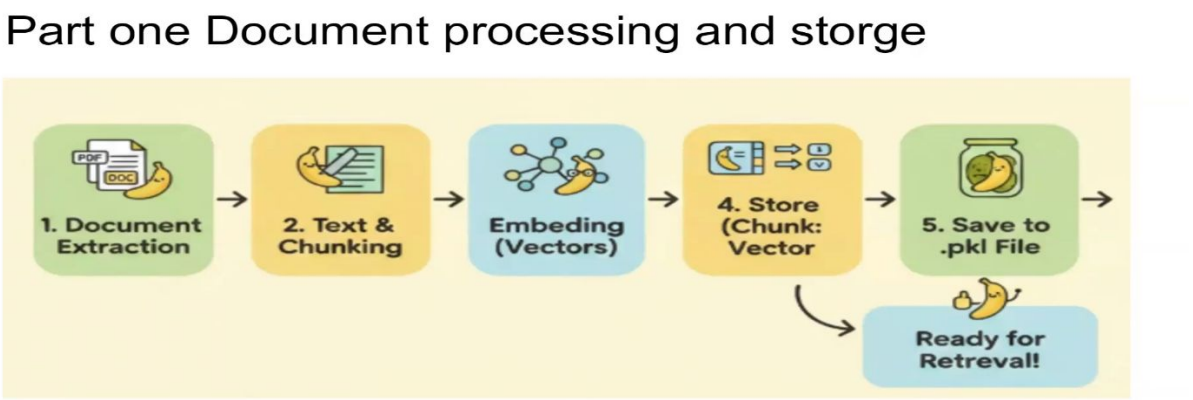
- **High Computational Cost**

  Training and running LLMs require significant computational resources, making them expensive and energy-intensive.

Ollama

A framework that allows you to run and use large language models locally on your own machine.

RAG Steps:



Part two **Retrieval and Generation**

RAG Implementation Steps Based on a txt file Done

Workflow

User Query

↓

Convert Query → Embedding

↓

Compare with Stored Embeddings

↓

Find Most Relevant Chunks

↓

Build Context

↓

Send Context + Question to LLM

↓

Generate Final Answer

1. Data Extraction

- Import libraries: ollama, numpy, pickle.
- Define the file path.
- Open the file.
- Read the file and store it in a variable.
- Define the chunk size.
- Split the text into chunks of specified character length.

```
import ollama
import numpy as np
import pickle

# Define file path
file_path = r"C:\Users\Muhammed Faris\OneDrive\Desktop\Ollama\lum_data.txt"


# Open and read file
with open(file_path, 'r', encoding='utf-8') as f:
    text = f.read()

# Define chunk size
chunk_size = 700

# Split text into chunks
chunks = [text[i:i + chunk_size] for i in range(0, len(text), chunk_size)]
```

2. Perform Embedding and Store in a List

- Create an empty list to store embeddings
- Generate embeddings for each text chunk using the nomic-embed-text model
- Store each embedding in the list

```
# Create empty list for embeddings
embeddings = []

# Generate embeddings for each chunk
for chunk in chunks:
    response = ollama.embeddings(
        model='nomic-embed-text',
        prompt=chunk
    )

    # Store embedding vector
    embeddings.append(response["embedding"])
```

3. Save Chunks and Embeddings in a Dictionary as Key-Value Pairs

- Create a dictionary to store text chunks and their embeddings
- Use "chunks" and "embeddings" as keys

```
# Store chunks and embeddings in a dictionary
data_to_save = {
    "chunks": chunks,
    "embeddings": embeddings
}
```

4. Save the Dictionary into a Pickle File

- Define the output pickle file name
- Open the file in write-binary (wb) mode

- Save the dictionary using pickle.dump()

```
# Define output pickle file
output_pickle = "knowledge.pkl"

# Save dictionary into pickle file
with open(output_pickle, 'wb') as f:
    pickle.dump(data_to_save, f)
```

5. Define a Function for Cosine Similarity

- Create a function that takes two vectors as input
- Compute the dot product of the vectors
- Divide by the product of their magnitudes (norms)

```
# Function to calculate cosine similarity
def cosine_similarity(v1, v2):
    return np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))
```

6. Final Step (Retrieval-Augmented Generation Pipeline)

This step performs:

- Loading stored embeddings
- Creating query embedding
- Finding most similar chunks
- Building a prompt
- Sending it to the LLM for response

```
# Number of top results to retrieve
k = 3

# Load pickle file
pickle_file = r"C:\Users\Muhammed Faris\OneDrive\Desktop\Ollama\knowledge.pkl"
```

```
with open(pickle_file, 'rb') as f:
    data = pickle.load(f)

# Extract stored data
chunks = data['chunks']
doc_embeddings = data['embeddings']

# Take user input
user_query = input("Enter the Query: ")

# Convert query to embedding
response = ollama.embeddings(
    model="nomic-embed-text",
    prompt=user_query
)

query_embedding = response["embedding"]

# Compute cosine similarity with all document embeddings
similarities = [
    cosine_similarity(query_embedding, emb)
    for emb in doc_embeddings
]

# Get top-k most relevant indices
top_k_indices = np.argsort(similarities)[-k:][::-1]

# Retrieve relevant chunks
retrieved_chunks = [chunks[i] for i in top_k_indices]

# Combine context
combined_context = "\n---\n".join(retrieved_chunks)

# Create prompt for LLM
prompt = f"""
Using the context below, answer the question. Only include information that directly answers the query.
```

```
Context:
{combined_context}

Question:
{user_query}

Strict Answer:
"""

# Generate response from LLM
output = ollama.generate(
    model="gemma3:1b",
    prompt=prompt,
    options={'temperature': 0.1}
)

print(output['response'])
```

Stable Diffusion

- Used for image generation.
- It is a repository/model available on the Hugging Face platform.
- Required libraries:
    - **Diffusers** – for Stable Diffusion pipelines.
    - **Transformers** – used for connecting text prompts with image generation.
    - **Accelerate** – used to improve processing speed and optimize performance.
    - **Safetensors** – used for safe and efficient model weight loading.
    - **Torch (PyTorch)** – the core deep learning framework
    - **Gradio** – used to create a simple user interface, similar to Streamlit.

Prompt Engineering

- **Prompt –** The input or question given to an AI model. It can include instructions, context, or a role that tells the AI how it should respond.
- **Prompt Engineering –** The practice of designing clear and effective prompts to get the desired output from Generative AI.

Steps in Prompt Engineering:

1. **Role** – Define who the AI should act as.
   Example: "Act as a Python developer."
2. **Instruction** – Clearly specify the task to be performed.
   Example: "Explain the code step by step."
3. **Examples** – Provide sample inputs and outputs if needed.
   Example: Showing a sample question and expected answer.
4. **Context** – Give background information related to the task.
   Example: "The audience is beginner-level students."
5. **Question** – Ask the final question or request.
   Example: "How can I optimize this program?".

Prompt Engineering Techniques:

- **Zero-Shot Prompting –** Asking the AI to perform a task without providing any examples.
- **Few-Shot Prompting** – Providing a few examples to help the AI understand the expected output format or style.
- **Chain of Thought Prompting –** Encouraging the AI to explain its reasoning step by step before giving the final answer.
- **Meta Prompting –** Creating prompts that help the AI generate or improve other prompts.
- **Self-Consistency –** Generating multiple reasoning paths and selecting the most consistent or accurate answer.

- **Prompt Chaining –** Breaking a complex task into multiple smaller prompts, where the output of one prompt becomes the input for the next.

Transformers

- Understanding the **Transformer Architecture** is important before attending an interview related to AI, NLP, or Generative AI.
  - Transfer Architecture by GeeksforGeeks
- **Transformers** is a framework/library available in the Hugging Face ecosystem.
- **Pipeline** – A high-level utility that combines multiple steps into a single process, making tasks like text generation, translation, summarization, and image processing easier to perform.